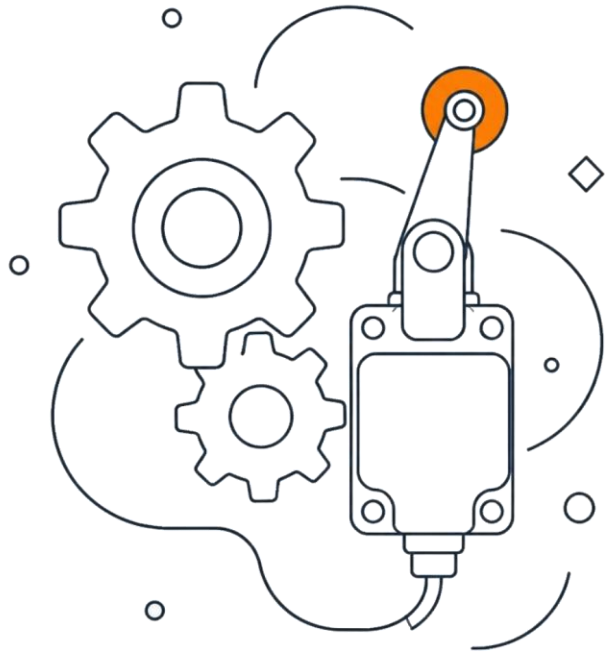
PostgreSQL · PL/pgSQL · Базы данных

Триггеры в PostgreSQL (PL/pgSQL)

Автоматическая реакция СУБД на события манипуляции данными: архитектура, фазы выполнения и практические сценарии.

BEFORE AFTER NEW / OLD



Концепция триггера

Триггер автоматически реагирует на изменение данных

Операции `INSERT`, `UPDATE` и `DELETE` запускают привязанную функцию без явного вызова со стороны приложения.

01

DML-запрос

Приложение выполняет `UPDATE products`, изменяя цену ноутбука с 65 000 до 70 000 ₽.

02

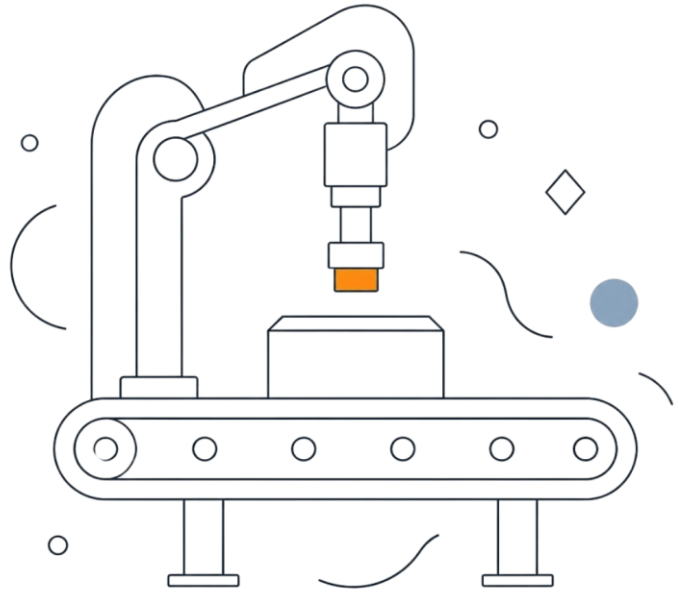
Срабатывание

СУБД перехватывает событие и передаёт строки `OLD` и `NEW` в триггерную функцию.

03

Аудит в таблице

Функция фиксирует изменение в служебном журнале `price_history` автоматически.



Сравнение механизмов выполнения

Функции, процедуры и триггеры запускаются по-разному

В отличие от функций и процедур, триггер невозможно вызвать напрямую по имени: его выполнением управляет ядро СУБД.

Тип объекта	Синтаксис вызова	Инициатор	Возврат значения
FUNCTION	`SELECT get_product_price(1);`	Явный вызов пользователем	Скаляр, таблица или VOID
PROCEDURE	`CALL update_product_price(1, 70000);`	Явный вызов CALL	Управление транзакциями
TRIGGER	`UPDATE products SET price = 70000;`	Событие изменения таблицы	RETURNS TRIGGER

Вопрос для аудитории: почему триггер надежнее явного вызова процедуры при ведении истории цен?

Триггерная функция задаёт действие, триггер — событие

В PostgreSQL сначала создаётся функция `RETURNS TRIGGER`, затем триггер на целевой таблице.

create_trigger_example.sql

```
CREATE OR REPLACE FUNCTION notify_insert()
RETURNS TRIGGER AS $$
BEGIN
  RAISE NOTICE 'Добавлен новый товар!';
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER product_insert_trigger
AFTER INSERT ON products FOR EACH ROW
EXECUTE FUNCTION notify_insert();
```

Функция (Что делать)

Не принимает аргументов в заголовке, возвращает `TRIGGER` и оперирует `NEW`.

Привязка (Когда делать)

Определяет фазу (`AFTER`), событие (`INSERT`), целевую таблицу и гранулярность (`FOR EACH ROW`).

Проверка: INSERT товара «Веб-камера», 4500, 10 → NOTICE «Добавлен новый товар!».

BEFORE проверяет и меняет строку; AFTER фиксирует последствия

Выбор момента срабатывания определяет доступные возможности воздействия на сохраняемые данные.

BEFORE (До записи на диск)

- **Валидация:** проверка ограничений и выброс исключений `RAISE EXCEPTION`.
- **Мутация строки:** изменение полей `NEW.col := value` до записи в таблицу.
- **Отмена строки:** возврат `RETURN NULL` отменяет операцию для текущей строки без прерывания всей транзакции.

AFTER (После фиксации в таблице)

- **Журналирование:** безопасная запись истории в связанные таблицы (аудит).
- **Каскадные действия:** обновление зависимых счетчиков, агрегатов и кэшей.
- **Возвращаемое значение:** результат `RETURN` полностью игнорируется построчным триггером.

Важно: `AFTER` не означает, что транзакция уже зафиксирована в базе (`COMMIT`). Если последующая операция внутри той же транзакции упадет с ошибкой, все изменения откатятся.

Специальные переменные

NEW и OLD показывают версии строки

В теле функции автоматически доступны структуры типа `RECORD`, состав которых зависит от типа DML-команды.

Операция	OLD	NEW	Назначение контекста
INSERT	NULL	Добавляемая	Новая строка до/после вставки
UPDATE	Исходная	Измененная	Сравнение старого и нового состояния
DELETE	Удаляемая	NULL	Данные строки перед её удалением



Пример UPDATE: `OLD.price = 65 000 ₽`, `NEW.price = 70 000 ₽`.

Вопрос: почему при DELETE нельзя использовать `NEW.name`?

Паттерн: Валидация

BEFORE INSERT запрещает отрицательную цену

check_product_price.sql

```
CREATE OR REPLACE FUNCTION
check_product_price()
RETURNS TRIGGER AS $$
BEGIN
 IF NEW.price < 0 THEN
  RAISE EXCEPTION
   'Цена не может быть отрицательной';
 END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

check_price_trigger

```
CREATE TRIGGER check_price_trigger
BEFORE INSERT ON products
FOR EACH ROW
EXECUTE FUNCTION check_product_price();
```

`INSERT INTO products VALUES ('Мышь', -5000, 5);` будет отклонена с ошибкой: `RAISE EXCEPTION` прерывает команду вставки.

`RAISE NOTICE` только выводит сообщение, но позволяет сохранить некорректную строку.

Паттерн: Нормализация

BEFORE UPDATE может исправить значение до сохранения

Вместо отклонения запроса триггер может переопределить поле в `NEW`, скорректировав данные на лету.

fix_quantity.sql

```
CREATE OR REPLACE FUNCTION fix_quantity()
RETURNS TRIGGER AS $$
BEGIN
 IF NEW.quantity < 0 THEN
  NEW.quantity := 0;
 END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER fix_quantity_trigger
BEFORE UPDATE ON products FOR EACH ROW
EXECUTE FUNCTION fix_quantity();
```

Результат автокоррекции

При выполнении `UPDATE products SET quantity = -10;` в таблице фактически сохранится значение `0`.

Архитектурная дискуссия

Исправлять или запрещать ошибку? Неявная подмена значений может скрыть баги в клиентском приложении, отправляющем отрицательные остатки.

Паттерн: Аудит изменений

История цен сохраняет каждое фактическое изменение

save_price_history.sql

```
CREATE OR REPLACE FUNCTION save_price_history()
RETURNS TRIGGER AS $$
BEGIN
  IF OLD.price IS DISTINCT FROM NEW.price THEN
    INSERT INTO price_history
      (product_id, old_price, new_price)
    VALUES (OLD.id, OLD.price, NEW.price);
  END IF; RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER price_history_trigger
AFTER UPDATE OF price ON products FOR EACH ROW
EXECUTE FUNCTION save_price_history();
```

`price_history`: product_id, old_price, new_price, changed_at. Идентификатор создаётся автоматически.

id	old	new	changed_at
1	65000	70000	10:15:20
1	70000	80000	11:42:05

Цена ноутбука: 65 000 → 70 000 → 80 000 ₽.

Проверка: UPDATE до 80 000 и SELECT из истории.

Управление триггерами и переменная TG_OP

DDL-команды управления

```
DROP TRIGGER check_price_trigger ON products;

ALTER TABLE products DISABLE TRIGGER check_price_trigger;
ALTER TABLE products ENABLE TRIGGER check_price_trigger;
```

Переменная TG_OP определяет вызвавшую операцию: `INSERT`, `UPDATE` или `DELETE`.

Отключение триггеров часто требуется при массовой миграции или начальной загрузке данных.